

GHC Dispatch Product Manual

GHC Dispatch is a Copilot-native agent orchestration platform. It keeps GitHub Copilot as the execution brain and adds a durable control plane around it: task scheduling, specialized agents, teams, approvals, policy checks, memory, automation, API access, MCP tools, Discord integration, and a VS Code extension.

This manual is the complete product reference. For short setup recipes, see [HOWTO.md](#). For the VS Code visual tour, see [VSCODE-EXTENSION-WALKTHROUGH.md](#).

Table of Contents

1. [Product Overview](#)
 2. [Core Architecture](#)
 3. [Installation and First Run](#)
 4. [Daemon and Runtime](#)
 5. [Command Line Reference](#)
 6. [Interactive TUI](#)
 7. [Task Model and Lifecycle](#)
 8. [Agents](#)
 9. [Teams](#)
 10. [Scheduler and Admission Control](#)
 11. [Policy and Approvals](#)
 12. [HTTP API Reference](#)
 13. [MCP Server](#)
 14. [VS Code Extension](#)
 15. [Discord Integration](#)
 16. [Memory and Wiki Knowledge](#)
 17. [Skills](#)
 18. [Automation and Events](#)
 19. [Browser Automation](#)
 20. [Model Switching](#)
 21. [Worktrees, Artifacts, and Multi-Repo Work](#)
 22. [Observability and Event Store](#)
 23. [Configuration Reference](#)
 24. [Development and Testing](#)
 25. [Operational Recipes](#)
 26. [Troubleshooting](#)
 27. [Glossary](#)
-

1. Product Overview

GHC Dispatch is designed for users who want GitHub Copilot quality plus workflow orchestration. Instead of replacing Copilot with a generic agent loop, Dispatch launches Copilot SDK sessions and coordinates them through its own scheduling, persistence, policy, and user-surface layers.

What Dispatch does

Dispatch lets you:

- Create work items as durable tasks.
- Assign tasks to specialist Copilot agents such as `@coder` , `@designer` , `@general-purpose` , or custom agents.
- Queue, prioritize, cancel, retry, pause, and recover tasks.
- Run multiple Copilot sessions concurrently with global, per-repo, and per-user limits.

- Coordinate multiple agents through teams.
- Use CLI, TUI, REST API, MCP, VS Code, and Discord surfaces.
- Capture task events, checkpoints, outputs, artifacts, and memory.
- Enforce approvals and policy checks around sensitive work.
- Schedule recurring jobs, webhook-triggered work, and event-driven automation.
- Use browser automation, skills, model switching, and cross-channel memory.

Why Copilot-native orchestration matters

Generic agent frameworks often use a model API as the "brain" and reimplement planning, context management, tool selection, and execution loops. Dispatch takes a different path:

```
Generic framework -> LLM provider API -> custom agent loop
Dispatch          -> Copilot SDK session -> native Copilot agent runtime
```

The Copilot-native design preserves Copilot's code-focused agent behavior while Dispatch adds the operational features needed to manage many tasks over time.

Primary user surfaces

Surface	Purpose
CLI (dispatch)	Scriptable task creation, status, daemon control, model switching
TUI	Interactive terminal workflow with slash commands and natural-language task creation
HTTP API	Automation, dashboards, integrations, VS Code extension backend
MCP Server	Exposes Dispatch as tools to MCP-compatible clients
VS Code Extension	Task, agent, skill, automation, approval, memory, and feature views
Discord Bot	Chat-based task creation, status, approvals, notifications, and memory

2. Core Architecture

Dispatch is organized into layered subsystems.

```
Surface Layer
  CLI / TUI / HTTP API / SSE / MCP / VS Code / Discord

Control Plane
  Task Manager / Scheduler / Policy Engine / Approval Manager / Event Bus

Execution Plane
  Copilot Adapter / Session Pool / Session Runner / Worktree Manager / Artifact Collector

Capability Layer
  Agents / Teams / Skills / MCP Servers / Browser Automation / Memory / Automation

Persistence
  SQLite database / event store / scheduler queue / wiki pages / task runtime settings
```

Source layout

Path	Purpose
src/index.ts	CLI entry point
src/daemon.ts	Daemon startup and subsystem wiring
src/config.ts	Environment-backed configuration
src/paths.ts	Persistent data paths under ~/.ghc-dispatch
src/control-plane/	Task state machine, scheduler, approvals, policy, event bus
src/execution/	Copilot sessions, worktrees, artifacts, models, reload, self-management
src/surfaces/	HTTP API, TUI, Discord bot
src/mcp/	MCP server and MCP app assets
src/store/	SQLite repositories and migrations
src/memory/	Conversation memory, episodic summaries, fact extraction
src/wiki/	Markdown wiki memory
src/skills/	Skill install/create/manage/search logic
src/automation/	Cron, webhook, GitHub, and proactive check-in automation
src/browser/	Playwright-powered browser automation
agents/	Built-in agent definitions
skills/	Bundled skills
plugin/	VS Code Agent Plugin assets
dispatch-vscode/	VS Code extension
docs/	Manual, HOWTO, and walkthrough documentation
tests/	Unit tests

Persistent data directory

Runtime data is stored under ~/.ghc-dispatch/ .

~/.ghc-dispatch/	
orchestrator.db	SQLite database
agents/	User custom agents
skills/	User/system skills
worktrees/	Isolated task worktrees
artifacts/	Captured diffs, logs, screenshots, outputs
wiki/	Markdown knowledge base
logs/	Runtime logs
task-runtime.json	Default session runtime config
execution-settings.json	Concurrency and idle-timeout settings

3. Installation and First Run

Prerequisites

Tool	Requirement
Node.js	Version 18 or newer
npm	Bundled with Node.js
Git	Any recent version
GitHub Copilot CLI / Copilot SDK access	Required for real Copilot execution
VS Code	Required only for the VS Code extension

Install from source

```
git clone https://github.com/boddev/ghc-dispatch.git
cd ghc-dispatch
npm install
npm run build
npm link
```

`npm link` makes the `dispatch` command available globally.

Verify the CLI

```
dispatch --version
dispatch --help
```

Start the daemon

```
dispatch --start
```

The daemon prints its data directory, API port, max session count, Copilot adapter mode, loaded agents, and health endpoints.

Run without a Copilot subscription

For local testing, use the mock adapter:

```
GHC MOCK COPILOT=1 dispatch --start
```

PowerShell:

```
$env:GHC MOCK COPILOT = "1"
dispatch --start
```

Create a first task

```
dispatch --create "Summarize this repository" --agent @general-purpose --priority normal
dispatch --list
dispatch --enqueue <task-id>
dispatch --events <task-id>
```

Create and immediately execute through the API:

```
curl -X POST http://localhost:7878/api/tasks \
-H "Content-Type: application/json" \
-d '{"title":"Summarize this repository","agent":"@general-purpose","priority":"normal"}'

curl -X POST http://localhost:7878/api/tasks/<task-id>/execute
```

4. Daemon and Runtime

The daemon is the long-running process that owns the scheduler, HTTP API, event stream, session pool, automation scheduler, memory services, Discord bot, browser engine, and persistence.

Start modes

Command	Behavior
dispatch --start	Start the daemon and keep it in the foreground
dispatch	Start or connect to daemon, then launch the TUI
VS Code extension	Can auto-start the daemon if dispatch.autoStartDaemon is enabled

Health endpoints

Endpoint	Purpose
GET /api/health	Basic daemon health, version, uptime
GET /api/stats	Task counts, scheduler/session stats, memory stats
GET /api/events/stream	Server-Sent Events stream

Hot reload, restart, and update

Command	API endpoint	Purpose
dispatch --reload	POST /api/reload	Reload agents and skills without restart
dispatch --restart	POST /api/restart	Spawn a replacement daemon and exit the old process
dispatch --update	POST /api/update	Pull updates, rebuild as needed, restart

Real Copilot vs mock Copilot

Mode	How to enable	Use case
Real Copilot	Default	Production and real coding tasks

5. Command Line Reference

The CLI format is:

```
dispatch --<command> [arguments] [options]
```

Running `dispatch` with no arguments opens the interactive TUI.

Commands

Command	Description
--create <title>	Create a new task
--status <task-id>	Show task details
--list	List tasks
--enqueue <task-id>	Queue a pending task
--cancel <task-id>	Cancel a task
--retry <task-id>	Retry a failed task
--events <task-id>	Show task event history
--stats	Show task and daemon statistics
--model	Show current default model and overrides
--model <name>	Switch default model
--model <name> --agent <agent>	Switch model for one agent
--models	List available models
--reload	Hot-reload agents and skills
--update	Update Dispatch
--restart	Restart the daemon
--start	Start the daemon
--version	Print CLI version
--help	Print help

Task creation flags

Flag	Description	Default
--agent <agent>	Agent handle, such as @coder	@general-purpose
--priority <level>	critical, high, normal, low	normal

--model <model>	One-time model override for this task	Agent/default model
--repo <path>	Target repository path	Current/default context
--description <text>	Longer task description	Empty
--dry-run	Resolve agent/model/workdir without creating	Disabled

List flags

Flag	Description	Default
--filter-status <status>	Filter by task status	All statuses
--limit <n>	Maximum rows returned	20

Cancel flags

Flag	Description
--reason <text>	Human-readable cancellation reason

CLI examples

Create a high-priority coding task:

```
dispatch --create "Fix the auth bug" --agent @coder --priority high --repo ~/dev/myapp
```

Preview task routing without creating:

```
dispatch --create "Refactor the billing module" --agent @coder --model gpt-5.5 --dry-run
```

List queued or running work:

```
dispatch --list --filter-status queued
dispatch --list --filter-status running --limit 50
```

Inspect a task:

```
dispatch --status 01KQ3ECDV5CJMS9DACYVJGM69K
dispatch --events 01KQ3ECDV5CJMS9DACYVJGM69K
```

Cancel with a reason:

```
dispatch --cancel 01KQ3ECDV5CJMS9DACYVJGM69K --reason "No longer needed"
```

Retry a failed task:

```
dispatch --retry 01KQ3ECDV5CJMS9DACYVJGM69K
```

Switch models:

```
dispatch --model
dispatch --model claude-opus-4.7
dispatch --model gpt-5.4 --agent @coder
dispatch --models
```

Control the daemon:

```
dispatch --reload
dispatch --restart
dispatch --update
```

6. Interactive TUI

The TUI is a readline-based terminal interface. It connects to the daemon, streams real-time events over SSE, and supports slash commands plus natural-language task creation.

Start it:

```
dispatch
```

If the daemon is not running, the TUI attempts to start it.

Slash commands

Command	Purpose
/help	Show available commands
/list [status]	List tasks, optionally filtered
/status <id>	Show task details
/create "title" [@agent] [!priority]	Create a task
/cancel <id>	Cancel a task
/retry <id>	Retry a failed task
/enqueue <id>	Queue a pending task
/execute <id>	Enqueue and dispatch immediately
/events <id>	Show task event history
/agents	List agents
/teams	List teams
/skills	List skills
/stats	Show system stats
/model [<name>]	Show or switch model
/models	List available models

/recall <topic>	Search cross-channel memory
/approvals	List pending approvals
/approve <id>	Approve an approval request
/reject <id>	Reject an approval request
/checkin	Trigger proactive check-in
/automation	List automation jobs
/reload	Hot-reload agents and skills
/restart	Restart daemon
/update	Update Dispatch
/quit	Exit TUI; daemon keeps running

Natural-language input

Anything that does not start with / becomes a task assigned to @general-purpose .

```
> Refactor the auth module to drop session cookies
```

This creates a task with that text as the title.

7. Task Model and Lifecycle

Tasks are durable units of work. A task records its title, description, agent, priority, repository, status, dependencies, retries, metadata, result, timestamps, and event history.

Task statuses

Status	Meaning
pending	Created but not queued
queued	Waiting for scheduler admission
running	Assigned to a Copilot session
paused	Interrupted and awaiting recovery choice
completed	Finished successfully
failed	Finished with error
cancelled	Cancelled by user/system

State transitions

```
pending -> queued -> running -> completed
                        | -> failed -> queued
                        | -> paused -> queued
                        | -> cancelled
```

```
pending -> cancelled
queued  -> cancelled
paused  -> cancelled
```

Terminal states are `completed` and `cancelled` .

Priority

Priority levels are:

- 1. `critical`
- 2. `high`
- 3. `normal`
- 4. `low`

The scheduler also applies aging boosts so old low-priority tasks are not starved forever.

Dependencies and DAGs

Tasks can depend on other tasks. Dispatch uses dependency information to form directed acyclic graphs (DAGs), validate cycles, execute ready nodes in topological order, and mark blocked nodes when dependencies fail.

Retries

Failed tasks can be retried up to the configured maximum. The default is 3 retries. Retry delay uses exponential backoff:

```
delay = RETRY_BACKOFF_MS * 2^attempt
```

Recovery after daemon restart

If the daemon stops while tasks are running, those tasks are marked `paused` on startup. Recovery options are:

Action	Effect
Resume	Re-queue against the existing worktree
Restart	Clear the worktree and start fresh
Abandon	Cancel the task with an optional reason

Recovery endpoints:

```
curl http://localhost:7878/api/tasks/<task-id>/recovery

curl -X POST http://localhost:7878/api/tasks/<task-id>/recovery \
-H "Content-Type: application/json" \
-d '{"action":"resume"}
```

8. Agents

Agents are specialist Copilot sessions. Each agent has metadata, a model, optional skills, optional tool constraints, optional MCP servers, and a system prompt.

Built-in agents

Agent	Default model	Role
@orchestrator	claude-sonnet-4.6	Plans, routes, delegates, synthesizes
@coder	gpt-5.4	Software engineering, debugging, implementation, tests
@designer	claude-opus-4.6	UI/UX, frontend components, accessibility
@general-purpose	auto	Research, docs, data processing, general tasks

Agent locations

Dispatch loads bundled agents and user agents:

Location	Purpose
agents/	Built-in repository agents
~/ .ghc-dispatch/agents/	User-created custom agents

Agent definition format

Agents are Markdown files with YAML frontmatter and a prompt body.

```
---
name: security-auditor
description: Reviews code for security vulnerabilities and risky patterns.
model: claude-opus-4.7
skills: []
tools:
  - read_file
  - grep
  - list_dir
mcpServers: []
domain: software-security
teamType: engineering
teamRoles:
  - reviewer
preferredTasks:
  - security review
antiTasks:
  - broad UI design
handoffStyle: Summarize findings with severity, evidence, and recommended fixes.
---
```

You are a senior application security reviewer.

Focus on injection, authentication, authorization, secrets, unsafe file access, and dependency risk. Cite evidence and avoid making code changes unless asked.

Frontmatter fields

Field	Required	Description
name	Yes	Display name and basis for agent handle

description	Yes	Short description shown in UIs
model	Yes	Default model for the agent
skills	No	Skills to load
tools	No	Tool allowlist; omit for default tool access
mcpServers	No	MCP servers available to the session
domain	No	Subject-area routing hint
teamType	No	Team category, such as engineering or design
teamRoles	No	Roles this agent can fill
preferredTasks	No	Tasks this agent should receive
antiTasks	No	Tasks this agent should avoid
handoffStyle	No	Expected completion summary style
leadershipStyle	No	How the agent behaves as a team lead

Create and reload an agent

```
# Save ~/.ghc-dispatch/agents/security-auditor.agent.md
dispatch --reload
dispatch --create "Audit the auth module" --agent @security-auditor --repo ~/dev/myapp
```

Generate an agent through the API:

```
curl -X POST http://localhost:7878/api/agents/generate \
-H "Content-Type: application/json" \
-d '{"description":"Senior accessibility reviewer for React applications"}'
```

9. Teams

Teams group a lead agent and member agents around a shared goal. The lead agent decomposes work, delegates bounded subtasks, and synthesizes the outputs.

How teams work

- 1. The lead receives the overall goal and reviews the team roster.
- 2. The lead creates a plan and delegates scoped work to member agents.
- 3. Members run independently as full Copilot sessions.
- 4. Members return handoff summaries.
- 5. The lead validates outputs, resolves conflicts, and produces a final response.

Built-in software development team

Role	Agent	Responsibility
Lead	@orchestrator	Planning, routing, sequencing, synthesis
Engineering	@coder	Code, tests, debugging, CI/CD

Design	@designer	UI/UX, styling, accessibility
Research/docs	@general-purpose	Documentation, research, specifications

When to use teams

Use teams for work that crosses domains, such as:

- Backend plus frontend plus documentation.
- Code audit plus findings report.
- Multi-component feature implementation.
- Large investigation with independent research streams.

Avoid teams for simple single-domain work. A single agent is faster and easier to reason about.

Run a team task

```
curl -X POST http://localhost:7878/api/teams/<team-id>/run \
-H "Content-Type: application/json" \
-d '{"title":"Add dark mode support","description":"Backend, UI, and docs","preApproved":true}'
```

Through MCP:

```
{ "teamId": "<team-id>", "title": "Add dark mode support" }
```

10. Scheduler and Admission Control

The scheduler decides when queued tasks can run. It prevents overload, preserves fairness, and respects configured concurrency limits.

Scheduler features

Feature	Description
Priority ordering	Critical work is admitted before high, normal, and low
Aging boost	Long-waiting tasks gain effective priority
Global concurrency	Caps total simultaneous Copilot sessions
Per-repo concurrency	Prevents one repo from consuming every session
Per-user concurrency	Provides fair sharing across users
Backpressure	Keeps tasks queued when capacity is exhausted
Durable queue	Scheduler queue is persisted in SQLite
Lease recovery	Interrupted leases can expire and be reacquired

Main concurrency setting

```
MAX_CONCURRENT_SESSIONS=4
```

The VS Code setting `dispatch.maxConcurrentSessions` can also sync this value to the daemon.

11. Policy and Approvals

Policy and approvals provide safety gates around sensitive work.

Policy engine

The policy engine evaluates rules before actions execute.

Rule type	Purpose	Example
repo_allowlist	Limit allowed repositories	Only allow org/app-*
tool_permission	Block risky tools	Deny deploy or destructive shell tools
approval_required	Require human approval	Require approval for production deploys
rate_limit	Limit action frequency	Max 10 tasks per hour
budget	Enforce quotas	Token/session budget per user

Example conceptual rule:

```
policyEngine.addRule({
  id: 'intern-safety',
  type: 'tool_permission',
  scope: {
    users: ['intern-alice'],
    agents: [],
    repos: [],
  },
  config: {
    deniedTools: ['git_push', 'deploy', 'rm'],
  },
});
```

Approval workflow

Approval requests are first-class objects with identity, task association, evidence, approvers, status, expiry, and event history.

```
{
  "id": "01KQ3...",
  "taskId": "01KQ2...",
  "type": "deployment",
  "description": "Deploy myapp to production",
  "evidence": ["diff.patch"],
  "approvers": ["admin"],
  "status": "pending",
  "expiresAt": "2026-04-26T12:00:00.000Z"
}
```

Approval commands

```
curl http://localhost:7878/api/approvals

curl -X POST http://localhost:7878/api/approvals/<approval-id>/approve \
-H "Content-Type: application/json" \
-d '{"decidedBy":"admin"}'

curl -X POST http://localhost:7878/api/approvals/<approval-id>/reject \
-H "Content-Type: application/json" \
-d '{"decidedBy":"admin","reason":"Not ready"}'
```

Pending approvals expire automatically and are recorded in the event store.

12. HTTP API Reference

The default API base URL is:

```
http://localhost:7878/api
```

Tasks

Method	Path	Description
POST	/api/tasks	Create a task
POST	/api/tasks/preview	Resolve task routing without persisting
GET	/api/tasks	List tasks
GET	/api/tasks/:id	Get task details
POST	/api/tasks/:id/enqueue	Queue a pending task
POST	/api/tasks/:id/execute	Enqueue and immediately dispatch
POST	/api/tasks/:id/cancel	Cancel a task
POST	/api/tasks/:id/cancellation-reason	Update reason on a cancelled task
POST	/api/tasks/:id/retry	Retry a failed task
GET	/api/tasks/:id/events	Get task event history
GET	/api/tasks/:id/subtasks	List child tasks
GET	/api/tasks/:id/recovery	Get paused-task recovery hints
POST	/api/tasks/:id/recovery	Resume, restart, or abandon paused task

Create a task:

```
curl -X POST http://localhost:7878/api/tasks \
-H "Content-Type: application/json" \
-d '{
  "title": "Add unit tests for auth",
  "description": "Cover token expiry and refresh flows",
}
```

```
"agent": "@coder",
"priority": "high",
"repo": "/path/to/repo"
}'
```

Preview task routing:

```
curl -X POST http://localhost:7878/api/tasks/preview \
-H "Content-Type: application/json" \
-d '{"title": "Refactor billing", "agent": "@coder", "model": "gpt-5.5"}'
```

Approvals

Method	Path	Description
GET	/api/approvals	List pending approvals
POST	/api/approvals/:id/approve	Approve a request
POST	/api/approvals/:id/reject	Reject a request

Agents and health

Method	Path	Description
GET	/api/agents	List loaded agents
GET	/api/agents/:name/content	Read raw .agent.md content
POST	/api/agents	Create or upsert an agent
POST	/api/agents/generate	Generate an agent from a role description
GET	/api/stats	System statistics
GET	/api/health	Health check
GET	/api/features	Feature catalog

Runtime configuration

Method	Path	Description
GET	/api/task-runtime/config	Read task runtime defaults
POST	/api/task-runtime/config	Update task runtime defaults
POST	/api/task-runtime/config/reset	Reset runtime defaults
GET	/api/execution/settings	Read execution settings
POST	/api/execution/settings	Update concurrency and idle timeout
POST	/api/integrations/workiq	Persist WorkIQ/Microsoft 365 settings

Models and chat

Method	Path	Description
GET	/api/models	List models, default, and overrides
GET	/api/models/current	Read current default model
POST	/api/models/switch	Switch default or per-agent model
POST	/api/models/reset	Clear agent model override
GET	/api/chat/model	Get Dispatch Chat model
POST	/api/chat/model	Switch Dispatch Chat model
POST	/api/chat	Send a Dispatch Chat message
GET	/api/chat/session	Inspect chat session metadata
DELETE	/api/chat/session	Reset chat session

Conversations and memory

Method	Path	Description
POST	/api/conversations	Log a cross-channel message
GET	/api/conversations	List recent messages
GET	/api/conversations/search	Search logged messages
GET	/api/conversations/threads	List conversation threads
GET	/api/conversations/thread/:channel/:threadId	Get a thread
POST	/api/memory/suggest	Suggest relevant context
POST	/api/memory/context	Build context for a conversation
GET	/api/memory/facts	Query extracted facts
GET	/api/memory/entities	List entities
GET	/api/memory/profile/:entity	Get an entity profile
GET	/api/memory/episodes	Query episodic summaries
GET	/api/memory/stats	Memory statistics

Skills

Method	Path	Description
GET	/api/skills	List skills
GET	/api/skills/:id	Get skill details
GET	/api/skills/:id/content	Read SKILL.md
POST	/api/skills/create	Create a system skill

POST	/api/skills/install/github	Install from GitHub
POST	/api/skills/install/registry	Install from skills.sh
POST	/api/skills/:id/enable	Enable a skill
POST	/api/skills/:id/disable	Disable a skill
DELETE	/api/skills/:id	Remove a skill

Automation and webhooks

Method	Path	Description
GET	/api/automation	List automation jobs
GET	/api/automation/:id	Get automation job
POST	/api/automation	Create automation job
POST	/api/automation/:id/enable	Enable job
POST	/api/automation/:id/disable	Disable job
POST	/api/automation/:id/run	Trigger job manually
DELETE	/api/automation/:id	Delete job
POST	/api/webhooks/:path	Trigger webhook job

Teams

Method	Path	Description
GET	/api/teams	List teams
POST	/api/teams	Create team
GET	/api/teams/:id	Get team
DELETE	/api/teams/:id	Delete team
POST	/api/teams/:id/run	Run task with full team

Browser automation

Method	Path	Description
POST	/api/browser/command	Execute natural-language browser command
POST	/api/browser/navigate	Navigate to URL
POST	/api/browser/click	Click by text or selector
POST	/api/browser/fill	Fill an input
POST	/api/browser/press	Press a key
POST	/api/browser/scroll	Scroll page

GET	/api/browser/page	Inspect current page
GET	/api/browser/screenshot	Capture screenshot
GET	/api/browser/text	Extract text
GET	/api/browser/status	Browser state

Daemon control and event stream

Method	Path	Description
POST	/api/reload	Hot-reload agents and skills
GET	/api/reload/status	Last reload result
POST	/api/restart	Restart daemon
POST	/api/update	Update Dispatch
GET	/api/checkin	Trigger proactive check-in
GET	/api/events/stream	SSE stream

13. MCP Server

Dispatch exposes itself as a Model Context Protocol server, making task orchestration available to MCP-compatible tools and chat clients.

MCP tools

Tool	Description
create_task	Create an orchestrated task
get_task	Get task status and details
list_tasks	List tasks with optional status filter
cancel_task	Cancel task
enqueue_task	Queue task
retry_task	Retry failed task
approve_task	Approve approval request
reject_task	Reject approval request
get_task_events	Get task event history
list_agents	List available agents
list_teams	List configured teams
run_team	Dispatch work to a team
get_stats	Get health and metrics
get_pending_approvals	List pending approvals

VS Code MCP configuration

Example `.vscode/mcp.json` :

```
{
  "servers": {
    "ghc-dispatch": {
      "type": "stdio",
      "command": "npx",
      "args": ["tsx", "/path/to/ghc-dispatch/src/mcp/server.ts"]
    }
  }
}
```

Then ask a compatible chat client to create or inspect tasks through the Dispatch MCP tools.

14. VS Code Extension

The VS Code extension lives in `dispatch-vscode/` . It provides a dedicated Dispatch activity-bar container with task, agent, skill, feature, automation, approval, chat, memory, and settings workflows.

For the screen-by-screen walkthrough, see [VSCODE-EXTENSION-WALKTHROUGH.md](#).

Install from source

```
cd dispatch-vscode
npm install
npm run compile
npm run package
code --install-extension dispatch-vscode-0.1.0.vsix
```

Views

View	Purpose
Tasks	Live task list grouped by status
Agents	Loaded agent definitions and models
Skills	User-installed and system-created skills
Features	Product feature catalog
Automation	Cron, webhook, and event jobs
Approvals	Pending approval requests

Key commands

Command palette command	Purpose
Dispatch: Create Task	Open create-task webview
Dispatch: Feature Catalog	Open feature catalog

Dispatch: Open Chat	Open Dispatch Chat
Dispatch: Open Settings	Open Dispatch settings
Dispatch: Configure Task Runtime	Edit runtime defaults
Dispatch: Configure Execution Settings	Edit concurrency and idle timeout
Dispatch: Configure WorkIQ	Configure Microsoft 365/WorkIQ integration
Dispatch: Show Stats	Show daemon stats
Dispatch: Recall Memory	Search memory
Dispatch: Switch Model	Switch default/chat/agent model
Dispatch: Reload Agents and Skills	Hot reload
Dispatch: Restart Daemon	Restart daemon
Dispatch: Update Dispatch	Update daemon
Dispatch: Create Agent	Generate or create an agent
Dispatch: Create Agent Team	Create a team
Dispatch: Install Skill	Install a skill from GitHub
Dispatch: Install Skill from skills.sh	Install registry skill
Dispatch: Create Automation Job	Create automation
Dispatch: Run Browser Command	Execute browser command
Dispatch: Memory Explorer	Browse memory

Create Task webview

The Create Task webview supports:

- Title and description.
- Agent or team selection.
- Priority.
- Optional model override.
- Repository and working directory.
- Pre-approved execution.
- Dry run preview.

Dry run calls `POST /api/tasks/preview` and displays resolved agent, model, priority, repository, and working directory without creating a task.

Task Detail webview

Task Detail shows:

- Status, agent, priority, retry count, repo, workdir.
- Actions: edit, cancel, delete, retry, enqueue, refresh.
- Plan files and Markdown documents.
- Artifacts.
- Approvals.

- Latest checkpoint.
- Event timeline.
- Cancellation reason editor for cancelled tasks.

Extension settings

Setting	Default	Purpose
dispatch.apiUrl	http://localhost:7878	Daemon API URL
dispatch.autoRefreshInterval	5000	Tree refresh interval in ms
dispatch.autoStartDaemon	true	Auto-run dispatch --start if daemon is down
dispatch.maxConcurrentSessions	4	Syncs session limit to daemon
dispatch.taskSessionIdleTimeoutMinutes	15	Syncs idle timeout to daemon

15. Discord Integration

Dispatch includes a Discord bot powered by `discord.js` . It can listen in configured channels, create tasks from commands or natural language, manage approvals, show status, and send notifications.

Setup

1. Create a bot in the Discord Developer Portal.
2. Enable Message Content Intent.
3. Invite the bot with `Send Messages` , `Read Message History` , and `Embed Links` .
4. Set environment variables.
5. Start Dispatch.

```
DISCORD_BOT_TOKEN=your-token
DISCORD_ALLOWED_CHANNELS=channel-id-1,channel-id-2
DISCORD_ADMIN_USERS=user-id-1,user-id-2
DISCORD_COMMAND_PREFIX=!dispatch
dispatch --start
```

If `DISCORD_ALLOWED_CHANNELS` is empty, all channels are allowed. Admin-only commands such as restart and update are gated by `DISCORD_ADMIN_USERS` .

Discord commands

Command	Description
<code>!dispatch create "title" [--agent @coder] [--priority high]</code>	Create task
<code>!dispatch list [--status running]</code>	List tasks
<code>!dispatch status <task-id></code>	Show task status
<code>!dispatch cancel <task-id></code>	Cancel task
<code>!dispatch retry <task-id></code>	Retry task
<code>!dispatch enqueue <task-id></code>	Queue task
<code>!dispatch approve <approval-id></code>	Approve request

<code>!dispatch reject <approval-id></code>	Reject request
<code>!dispatch agents</code>	List agents
<code>!dispatch skills</code>	List skills
<code>!dispatch stats</code>	Show stats
<code>!dispatch recall <topic></code>	Search memory
<code>!dispatch model</code>	Show current model
<code>!dispatch model <model></code>	Switch default model
<code>!dispatch model <model> --agent @coder</code>	Switch agent model
<code>!dispatch help</code>	Help

Natural-language task creation

Mention the bot or send it a DM:

```
@dispatch fix the login bug in the auth module
```

Dispatch creates a task and can include related context from cross-channel memory.

Notifications

The bot can notify Discord when:

- A Discord-created task completes.
- A Discord-created task fails.
- Approval is requested.
- Relevant memory context is found.

16. Memory and Wiki Knowledge

Dispatch has two complementary memory systems:

1. A structured conversation/fact memory in SQLite.
2. A Markdown wiki stored under `~/.ghc-dispatch/wiki`.

Conversation log

Messages from CLI, Discord, VS Code, API, GitHub, and other channels can be logged with:

- Channel.
- Thread ID.
- Speaker identity.
- Speaker type.
- Content.
- Timestamp.

Search examples:

```
curl "http://localhost:7878/api/conversations/search?q=JWT"
curl "http://localhost:7878/api/conversations/search?q=deployment&channel=discord"
```

```
curl "http://localhost:7878/api/conversations/threads?channel=cli"
```

Episodic memory

An episodic writer summarizes idle conversation threads into dated wiki pages. Summaries include:

- Speaker-attributed digest.
- Topics.
- Entities.
- Decisions.
- Cross-links.

Search:

```
curl "http://localhost:7878/api/memory/episodes?q=deployment"
curl "http://localhost:7878/api/memory/episodes?date=2026-04-26"
```

Proactive fact extraction

Dispatch extracts facts from messages about people, projects, tools, preferences, and work patterns.

Category	Example
Preference	"prefers TypeScript"
Identity	"works at Microsoft"
Tooling	"uses Vitest"
Project	"myapp deploys to Vercel"
Work pattern	"starts at 9am"

Query facts:

```
curl "http://localhost:7878/api/memory/facts?q=TypeScript"
curl "http://localhost:7878/api/memory/entities"
curl "http://localhost:7878/api/memory/profile/alice"
```

Context suggestions

Dispatch can suggest relevant context for a new message:

```
curl -X POST http://localhost:7878/api/memory/suggest \
-H "Content-Type: application/json" \
-d '{"message":"Fix the JWT token expiry","channel":"cli"}'
```

Build a context block:

```
curl -X POST http://localhost:7878/api/memory/context \
-H "Content-Type: application/json" \
-d '{"message":"Set up the project","speakers":["alice"],"channel":"vscode"}'
```


Wiki memory

The wiki stores Markdown pages with YAML frontmatter and wiki-style links.

Capabilities include:

- Entity pages for people, projects, and concepts.
- Tags.
- Search.
- Cross-links.
- Remember/forget operations.
- Context index generation.

Wiki files live in:

```
~/ .ghc-dispatch/wiki/pages/
```

17. Skills

Skills are instruction packages that teach agents how to use external systems and tools. They are stored as `SKILL.md` files.

Skill categories

Category	Description
User-installed	Manually installed from a registry or GitHub repo
System-created	Created by Dispatch when learning a new tool
Registry	Installed from skills.sh
GitHub	Installed from any public GitHub repository
Bundled	Included in the repository under <code>skills/</code>

Bundled skills

Skill	Purpose
google-workspace	Gmail, Calendar, Drive workflows through <code>gogcli</code>
microsoft-365	Outlook, Calendar, OneDrive workflows through Microsoft Graph

Install skills

From skills.sh:

```
curl -X POST http://localhost:7878/api/skills/install/registry \  
-H "Content-Type: application/json" \  
-d '{"name":"kubernetes"}'
```

From GitHub:

```
curl -X POST http://localhost:7878/api/skills/install/github \  
-H "Content-Type: application/json" \  
-d '{"name":"kubernetes"}'
```

```
-d '{"repoUrl":"https://github.com/owner/skill-repo"}'
```

Create locally:

```
curl -X POST http://localhost:7878/api/skills/create \
-H "Content-Type: application/json" \
-d '{"name":"docker-compose","description":"Manage Docker Compose apps","instructions":"Use docker compose ..."}'
```

Manage skills

```
curl http://localhost:7878/api/skills
curl http://localhost:7878/api/skills/kubernetes/content
curl -X POST http://localhost:7878/api/skills/kubernetes/disable
curl -X POST http://localhost:7878/api/skills/kubernetes/enable
curl -X DELETE http://localhost:7878/api/skills/kubernetes
```

18. Automation and Events

Automation jobs create or trigger work from schedules, webhooks, and internal events.

Trigger types

Type	Description	Example
Cron	Periodic execution	Run tests every hour
Webhook	HTTP-triggered execution	Run checks when CI notifies Dispatch
Event	React to Dispatch events	Create review task after completion

Action types

Action	Description
create_task	Create a task
run_command	Execute command
http_request	Call external URL
log	Log message

Cron job example

```
curl -X POST http://localhost:7878/api/automation \
-H "Content-Type: application/json" \
-d '{
  "name": "Nightly tests",
  "type": "cron",
  "schedule": "daily",
  "action": "create_task",
  "actionConfig": {
    "title": "Run nightly test suite",
```

```
    "agent": "@coder",
    "priority": "high"
  }
}'
```

Webhook example

```
curl -X POST http://localhost:7878/api/automation \
-H "Content-Type: application/json" \
-d '{
  "name": "CI complete",
  "type": "webhook",
  "webhookPath": "ci-notify",
  "action": "create_task",
  "actionConfig": {
    "title": "CI passed - run integration tests",
    "agent": "@coder"
  }
}'

curl -X POST http://localhost:7878/api/webhooks/ci-notify \
-H "Content-Type: application/json" \
-d '{"commit": "abc123", "branch": "main"}'
```

Event-triggered example

```
curl -X POST http://localhost:7878/api/automation \
-H "Content-Type: application/json" \
-d '{
  "name": "Auto-review",
  "type": "event",
  "eventType": "task.completed",
  "action": "create_task",
  "actionConfig": {
    "title": "Review completed work",
    "agent": "@coder"
  }
}'
```

Supported internal events

Common event types include:

- `task.created`
- `task.queued`
- `task.started`
- `task.output`
- `task.checkpoint`
- `task.completed`
- `task.failed`
- `task.cancelled`
- `task.paused`

- `task.resumed`
- `task.retrying`
- `approval.requested`
- `approval.decided`
- `session.created`
- `session.destroyed`
- `artifact.captured`

GitHub webhooks

Point a GitHub repository webhook at:

```
http://your-dispatch-host:7878/api/webhooks/github
```

Supported GitHub events include:

GitHub event	Dispatch behavior
Pull request opened	Create review task
Issue opened	Create investigation task; bug labels can raise priority
Check run failed	Create high-priority fix task
Workflow run failed	Create high-priority fix task
Push	Log to memory
Issue comment	Log to memory
Release published	Log to memory

Use `GITHUB_WEBHOOK_SECRET` to validate incoming GitHub webhook signatures.

Proactive check-ins

Dispatch periodically checks for conditions that should be surfaced:

Check	Example
Failed tasks	Failed but not retried
Expiring approvals	Approval expires soon
Stuck tasks	Running too long
Stale queue	Waiting too long
Recurring failures	Same agent failing repeatedly
Daily summary	Conversation and task activity

Trigger manually:

```
curl http://localhost:7878/api/checkin
```

19. Browser Automation

Dispatch includes a Playwright-powered browser engine that can execute natural-language browser commands and deterministic API calls.

Natural-language commands

```
curl -X POST http://localhost:7878/api/browser/command \
-H "Content-Type: application/json" \
-d '{"command": "go to github.com"}'

curl -X POST http://localhost:7878/api/browser/command \
-H "Content-Type: application/json" \
-d '{"command": "click Sign in"}'

curl -X POST http://localhost:7878/api/browser/command \
-H "Content-Type: application/json" \
-d '{"command": "fill alice@example.com in email"}'

curl -X POST http://localhost:7878/api/browser/command \
-H "Content-Type: application/json" \
-d '{"command": "read the page"}'
```

Supported patterns

Pattern	Example
go to <url>	go to github.com
open <url>	open docs.github.com
click <text>	click Sign in
fill <value> in <field>	fill alice@example.com in email
set <field> to <value>	set email to alice@example.com
scroll down / scroll up	scroll down
go back	go back
search for <query>	search for playwright docs
read the page	read page content
screenshot	take a screenshot
press <key>	press Enter

Direct browser API

Use direct endpoints when you need deterministic behavior:

```
curl -X POST http://localhost:7878/api/browser/navigate \
-H "Content-Type: application/json" \
-d '{"url": "https://github.com"}'
```

```
curl -X POST http://localhost:7878/api/browser/click \
  -H "Content-Type: application/json" \
  -d '{"text": "Sign in"}'

curl http://localhost:7878/api/browser/page
curl http://localhost:7878/api/browser/text
curl http://localhost:7878/api/browser/status
```

20. Model Switching

Dispatch supports runtime model switching without editing agent files or restarting the daemon.

Resolution order

Model selection uses this priority order:

1. Per-task override.
2. Per-agent runtime override.
3. Agent definition `model` field.
4. Global default model.

CLI

```
dispatch --model
dispatch --model claude-opus-4.7
dispatch --model gpt-5.4 --agent @coder
dispatch --models
dispatch --create "Complex analysis" --agent @coder --model gpt-5.5
```

API

```
curl http://localhost:7878/api/models

curl -X POST http://localhost:7878/api/models/switch \
  -H "Content-Type: application/json" \
  -d '{"model": "claude-opus-4.7"}'

curl -X POST http://localhost:7878/api/models/switch \
  -H "Content-Type: application/json" \
  -d '{"model": "gpt-5.4", "agent": "@coder"}'

curl -X POST http://localhost:7878/api/models/reset \
  -H "Content-Type: application/json" \
  -d '{"agent": "@coder"}'
```

Discord

```
!dispatch model
!dispatch model claude-opus-4.7
!dispatch model gpt-5.4 --agent @coder
```

Example model IDs

Model	Provider	Tier
claude-sonnet-4.6	Anthropic	Standard
claude-opus-4.7	Anthropic	Premium
gpt-5.5	OpenAI	Premium
gpt-5.4	OpenAI	Standard
gpt-5.4-mini	OpenAI	Free/mini
gemini-2.5-pro	Google	Standard

Run `dispatch --models` for the full list available in the current build.

21. Worktrees, Artifacts, and Multi-Repo Work

Worktree isolation

Dispatch creates isolated git worktrees for task execution. This prevents running tasks from modifying your main working tree or colliding with each other.

Worktrees are stored under:

```
~/ .ghc-dispatch/worktrees/
```

Retention is controlled by:

```
WORKTREE_RETENTION_MINUTES=60
```

Artifacts

The artifact collector captures outputs such as:

- Diffs.
- Logs.
- Files.
- Screenshots.
- Task output summaries.

Artifacts are stored under:

```
~/ .ghc-dispatch/artifacts/
```

They are surfaced in task detail views and task event history.

Multi-repo coordination

Dispatch can coordinate tasks spanning multiple repositories. Each repository gets an isolated worktree for the task.

Conceptual TypeScript usage:

```
import { MultiRepoCoordinator } from './src/execution/multi-repo.js';

const coordinator = new MultiRepoCoordinator(baseDir, worktreeManager);

const workspace = await coordinator.setupWorkspace('feature-x', [
  { url: 'https://github.com/org/frontend.git', name: 'frontend', defaultBranch: 'main' },
  { url: 'https://github.com/org/backend.git', name: 'backend', defaultBranch: 'main' },
]);

const worktrees = await coordinator.createWorktrees('feature-x', 'task-123');
await coordinator.cleanupWorktrees('feature-x', 'task-123');
```

22. Observability and Event Store

Dispatch records state changes as immutable events. Events power task debugging, task detail views, audit trails, SSE updates, automation triggers, and recovery.

Event types

Event	Description
task.created	Task created
task.queued	Task entered queue
task.started	Execution began
task.output	Agent emitted output
task.checkpoint	Checkpoint saved
task.completed	Task completed
task.failed	Task failed
task.cancelled	Task cancelled
task.paused	Task paused for recovery
task.resumed	Task resumed
task.retrying	Retry started
approval.requested	Approval requested
approval.decided	Approval approved or rejected
session.created	Copilot session started
session.destroyed	Copilot session ended
artifact.captured	Artifact captured

Query events


```
dispatch --events <task-id>
curl http://localhost:7878/api/tasks/<task-id>/events
curl http://localhost:7878/api/events/stream
```

Retention

Event/log retention is configured with:

```
LOG_RETENTION_DAYS=30
```

23. Configuration Reference

Configuration is loaded from environment variables and persisted settings files.

Environment variables

Variable	Default	Purpose
COPILOT_MODEL	claude-sonnet-4.6	Global default model
COPILOT_DEFAULT_REMOTE	1	Prepend /remote to Copilot CLI sessions when true
API_PORT	7878	HTTP API port
DISPATCH_API_KEY	unset	Optional API key for API access
GITHUB_WEBHOOK_SECRET	unset	Secret for validating GitHub webhooks
MAX_CONCURRENT_SESSIONS	4	Global concurrent Copilot session limit
MAX_RETRIES_PER_TASK	3	Default retry count
RETRY_BACKOFF_MS	2000	Retry base backoff in milliseconds
WORKTREE_RETENTION_MINUTES	60	Completed worktree retention
LOG_RETENTION_DAYS	30	Event/log retention
DISCORD_BOT_TOKEN	unset	Discord bot token
DISCORD_ALLOWED_CHANNELS	empty	Comma-separated allowed Discord channel IDs
DISCORD_ADMIN_USERS	empty	Comma-separated Discord user IDs allowed to run admin commands
DISCORD_COMMAND_PREFIX	!dispatch	Discord command prefix
GHC MOCK_COPILOT	0	Use mock Copilot adapter when 1

API key behavior

When DISPATCH_API_KEY is set, API clients should send the key according to the server's accepted auth behavior. Use this for local or trusted-network deployments that need a simple access gate.

Runtime settings files

File	Purpose
------	---------

~/ghc-dispatch/task-runtime.json	Task runtime defaults
~/ghc-dispatch/execution-settings.json	Concurrency and idle-timeout settings

These can be managed through the REST API or the VS Code extension settings flows.

24. Development and Testing

Root package scripts

Script	Description
npm run build	Compile TypeScript
npm run dev	Start daemon with tsx --watch
npm start	Start daemon
npm run cli	Run CLI entry point through tsx
npm test	Run Vitest test suite
npm run test:coverage	Run focused coverage suite
npm run test:watch	Watch tests
npm run typecheck	Type-check without emit
npm run lint	Type-check without emit

Recommended validation

```
npm run typecheck
npm run build
npm run test -- --maxWorkers=1
```

For the VS Code extension:

```
cd dispatch-vscode
npm run compile
```

Technology stack

Component	Technology
Runtime	Node.js
Language	TypeScript ESM
Execution	GitHub Copilot SDK
HTTP	Express 5
Database	SQLite with better-sqlite3
Validation	Zod

MCP	@modelcontextprotocol/sdk
Discord	discord.js
Browser	Playwright
Testing	Vitest

25. Operational Recipes

Start Dispatch for local development

```
npm install
npm run build
npm link
GHC MOCK COPILOT=1 dispatch --start
```

Create, execute, and watch a task

```
dispatch --create "Add tests for scheduler retry behavior" --agent @coder --priority high
dispatch --list
dispatch --enqueue <task-id>
dispatch --events <task-id>
```

Create and execute in one API flow

```
TASK_ID=$(curl -s -X POST http://localhost:7878/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Add docs for model switching","agent":"@general-purpose"}' | jq -r '.id')

curl -X POST http://localhost:7878/api/tasks/$TASK_ID/execute
```

Recover paused work

```
curl http://localhost:7878/api/tasks/<task-id>/recovery

curl -X POST http://localhost:7878/api/tasks/<task-id>/recovery \
  -H "Content-Type: application/json" \
  -d '{"action":"resume"}'
```

Install and use the VS Code extension

```
cd dispatch-vscode
npm install
npm run compile
npm run package
code --install-extension dispatch-vscode-0.1.0.vsix
```

Reload VS Code, open the Dispatch activity-bar icon, and let the extension auto-start the daemon or start it manually with `dispatch --start` .

Add a custom agent

```
mkdir -p ~/.ghc-dispatch/agents
# Create ~/.ghc-dispatch/agents/security-auditor.agent.md
dispatch --reload
dispatch --create "Review auth for security bugs" --agent @security-auditor
```

Trigger a proactive check-in

```
curl http://localhost:7878/api/checkin
```

26. Troubleshooting

`dispatch` command not found

Run from the repo root:

```
npm run build
npm link
dispatch --version
```

On Windows, run the terminal as Administrator if global linking fails.

Daemon does not start

Check:

- Node.js version is 18 or newer.
- Dependencies are installed with `npm install` .
- Port `7878` is not already occupied, or set `API_PORT` .
- Copilot subscription/auth is available, or use `GHC MOCK COPILOT=1` .

VS Code extension does not connect

Check:

- `dispatch --version` works in a terminal.
- `dispatch --start` is running or `dispatch.autoStartDaemon` is enabled.
- `dispatch.apiUrl` matches the daemon URL.
- `curl http://localhost:7878/api/health` succeeds.

Tasks stay queued

Check:

- `MAX_CONCURRENT_SESSIONS` .
- Running task count from `dispatch --stats` .
- Per-repo or per-user limits.
- Whether approvals are pending.
- Whether a paused task needs recovery.

Discord bot does not respond

Check:

- `DISCORD_BOT_TOKEN` is set.
- Message Content Intent is enabled.
- Bot has channel permissions.
- Channel is included in `DISCORD_ALLOWED_CHANNELS` .
- Command prefix matches `DISCORD_COMMAND_PREFIX` .

GitHub webhook tasks are not created

Check:

- Webhook URL is `/api/webhooks/github` .
- Content type is `application/json` .
- Selected events include PRs, issues, checks, or workflow runs.
- `GITHUB_WEBHOOK_SECRET` matches the webhook secret if configured.

Browser automation fails

Check:

- Playwright dependencies are installed.
- The page is reachable.
- Selectors/text are stable.
- Use direct endpoints for deterministic control when natural-language commands are ambiguous.

27. Glossary

Term	Meaning
Agent	Specialist Copilot session configured with a prompt, model, skills, and tools
Approval	Human decision gate required before sensitive work continues
Artifact	Captured output such as diff, log, file, or screenshot
Control plane	Task manager, scheduler, policy, approvals, and event bus
DAG	Directed acyclic graph of dependent tasks
Daemon	Long-running Dispatch process
Event store	Append-only record of task/session/approval/artifact events
MCP	Model Context Protocol, used to expose Dispatch tools to compatible clients
Session	Copilot SDK execution context used to run a task
Skill	<code>SKILL.md</code> instruction package that teaches agents external capabilities
Team	Lead agent plus member agents working toward a shared goal
TUI	Interactive terminal UI launched by <code>dispatch</code>
Wiki	Markdown knowledge base under <code>~/.ghc-dispatch/wiki</code>
Worktree	Isolated git working directory for task execution

Related Documentation

- [README.md](#) - Product overview and documentation index.
- [HOWTO.md](#) - Practical setup and workflow recipes.
- [VSCODE-EXTENSION-WALKTHROUGH.md](#) - Visual VS Code extension walkthrough.
- [agents/teams.md](#) - Team operating model and examples.